

Behavioral Modeling in VERILOG

Dr. Chandan Karfa

Department of Computer Science and Engineering



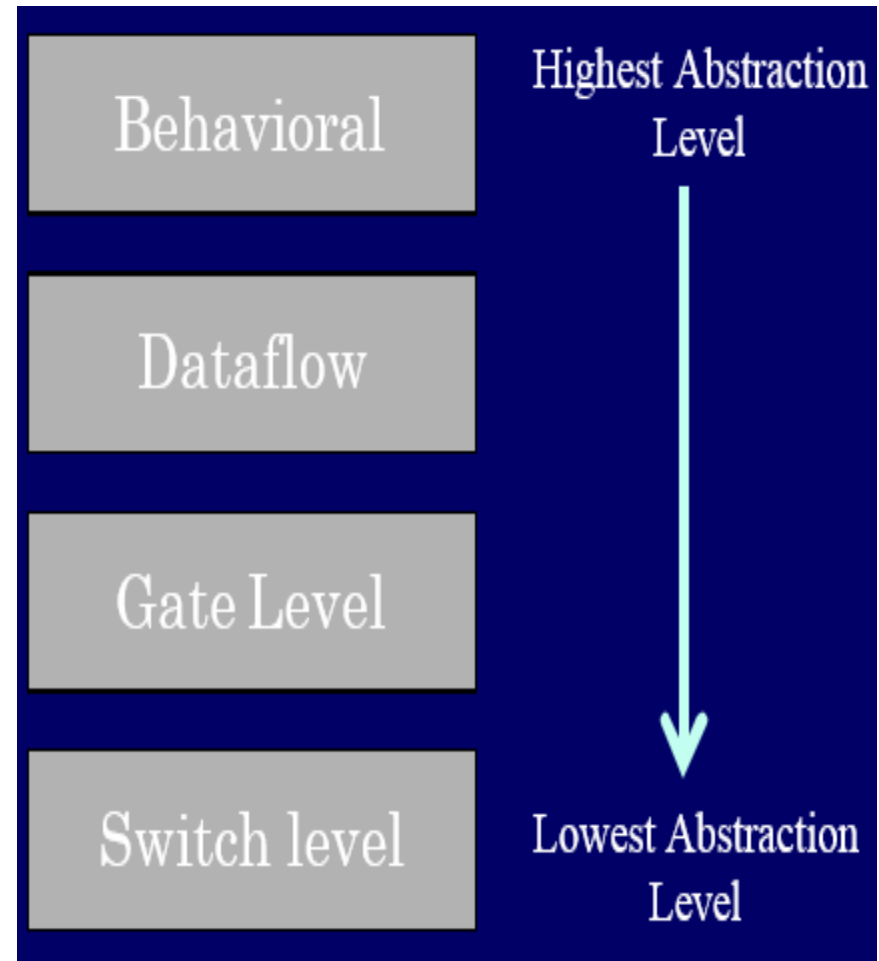
भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI



- S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Pearson, 2nd Ed, 2003.

Modeling Techniques

- Behavioral Modeling
- Data-flow Modeling
- Gate level Modeling
- Switch level Modeling



Behavioral Modeling

- Used to model the behavior of a design without describing its actual hardware structure.
- Procedural blocks are the basic components for behavioral modeling.

```
initial  
begin  
    ... imperative statements ...  
end
```

- Runs when simulation starts
- Terminates when control reaches the end
- Good for providing stimulus

```
always  
begin  
    ... imperative statements ...  
end
```

- Runs when simulation starts
- Restarts when control reaches the end
- Good for modeling / specifying hardware

Initial statement

```
module stimulus;
  reg x,y, a,b, m;
  initial
    m = 1'b0; //single statement;
  initial
    begin
      #5 a = 1'b1;
      #25 b = 1'b0;
    end
  initial
    begin
      #10 x = 1'b0;
      #25 y = 1'b1;
    end
  initial
    #50 $finish;
endmodule
```

time statement executed

```
0 m = 1'b0;
5 a = 1'b1;
10 x = 1'b0;
30 b = 1'b0;
35 y = 1'b1;
50 $finish;
```

The initial blocks are typically used for initialization, monitoring, waveforms and other processes that must be executed only once during the entire simulation run.

Always statement

- always statement starts at time 0 and executes the statements in the always block continuously in a looping fashion.

```
module clock_gen (output reg clock);  
//Initialize clock at time zero  
initial  
    clock = 1'b0;  
//Toggle clock every half-cycle (time period = 20)  
always  
    #10 clock = ~clock;  
initial  
    #1000 $finish;  
endmodule
```

Initial and Always Statements

- Procedural blocks are like concurrent processes. Statements in a block are executed sequentially, but all within one unit of simulated time. (unless delay is specified)
- All blocks execute in parallel.
- initial block – Executes only once. It is **NOT** synthesizable. Used in test benches.
- always block - Executes repeatedly. It must have timing control, otherwise it become INFINITE LOOPS.
- ***Procedural assignments within initial/always:***
 - **LHS** must be reg & cannot be net.
 - **RHS** can be net/reg.

Event Control

- **Edge-triggered Event control**

@ (posedge CLK) //positive edge of CLK
Curr_state = Next_state ;

@ negedge	@ posedge
1 → x	0 → x
1 → z	0 → z
1 → 0	0 → 1
x → 0	x → 1
z → 0	z → 1

- **Level-triggered Event control**

@ (A or B) //change in values of A or B
out = A & B

//Gate-level description of 4-to-1- line multiplexer

```
module mux4x1 (I0, I1, I2, I3, s1, s0, y);
```

```
    input I0, I1, I2, I3, s1, s0;
```

```
    output y;
```

```
    //intermediate connections
```

```
    wire NOT_s1, NOT_s0 ;
```

```
    wire y0, y1, y2, y3 ;
```

```
    not n0 (NOT_s0, s0) ;
```

```
    not n1 (NOT_s1, s1) ;
```

```
    and a0 (y0, I0, NOT_s1, NOT_s0) ;
```

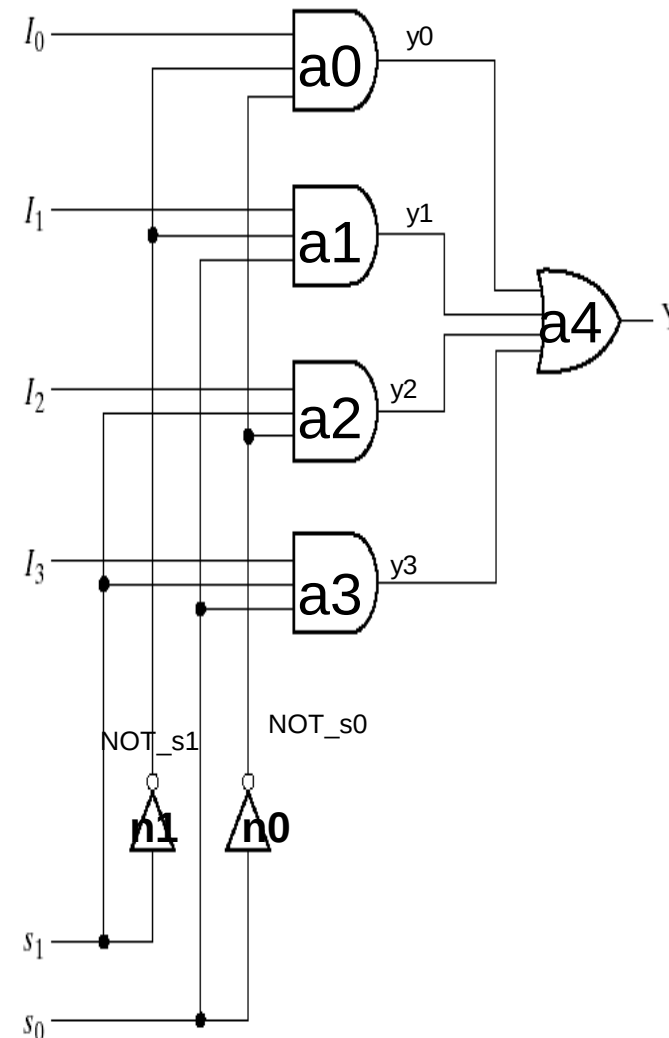
```
    and a1 (y1, I1, NOT_s1,      s0) ;
```

```
    and a2 (y2, I2,      s1, NOT_s0) ;
```

```
    and a3 (y3, I3,      s1,      s0) ;
```

```
    or a4 (y, y3, y2, y1, y0) ;
```

```
endmodule
```



s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

//Data-flow description of 4-to-1- line multiplexer

```
module mux4x1 (I0, I1, I2, I3, s1, s0, y);
```

```
    input I0, I1, I2, I3, s1, s0;
```

```
    output y;
```

```
    assign y = (~s1 & ~s0 & I0) |  
               (~s1 &  s0 & I1) |  
               ( s1 & ~s0 & I2) |  
               ( s1 &  s0 & I3);
```

```
endmodule
```

//Using conditional operator

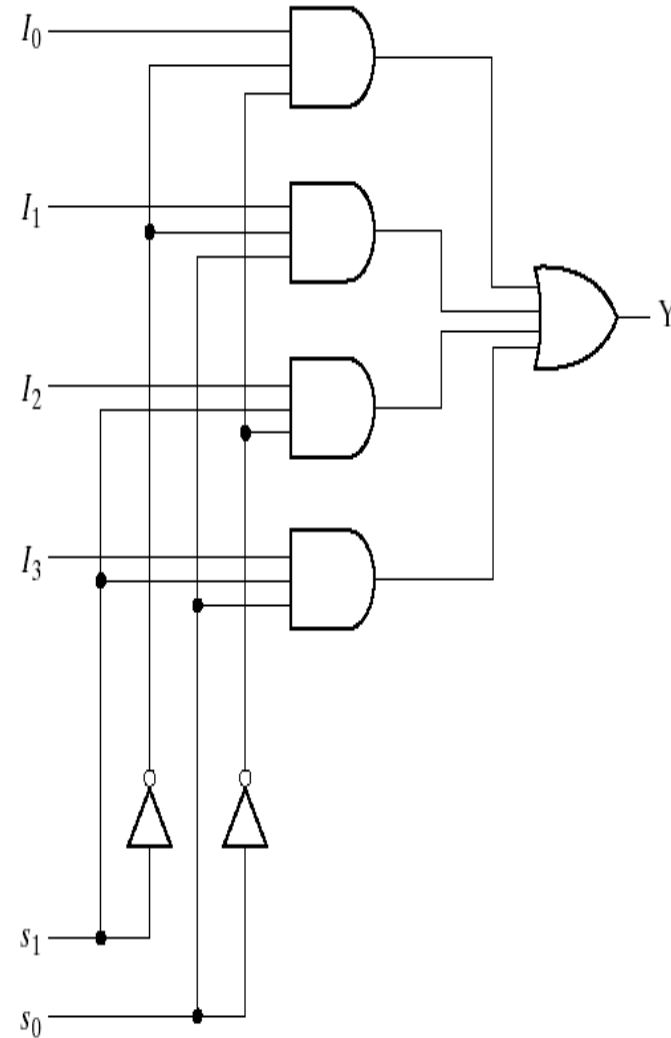
```
module mux2x1 (I0, I1, sel, y);
```

```
    input I0, I1, sel;
```

```
    output y;
```

```
    assign y = sel ? I1 : I0 ;
```

```
endmodule
```



s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

//Behavioral description of 4-to-1- line multiplexer

```
module mux4x1_bhv (I0, I1, I2, I3, select, y);
```

```
    input I0, I1, I2, I3;
```

```
    input [1:0] select;
```

```
    output y;
```

```
    reg y;
```

```
    always @ (I0 or I1 or I2 or I3 or select)
```

```
    begin
```

```
        case (select)
```

```
            2'b00: y = I0;
```

```
            2'b01: y = I1;
```

```
            2'b10: y = I2;
```

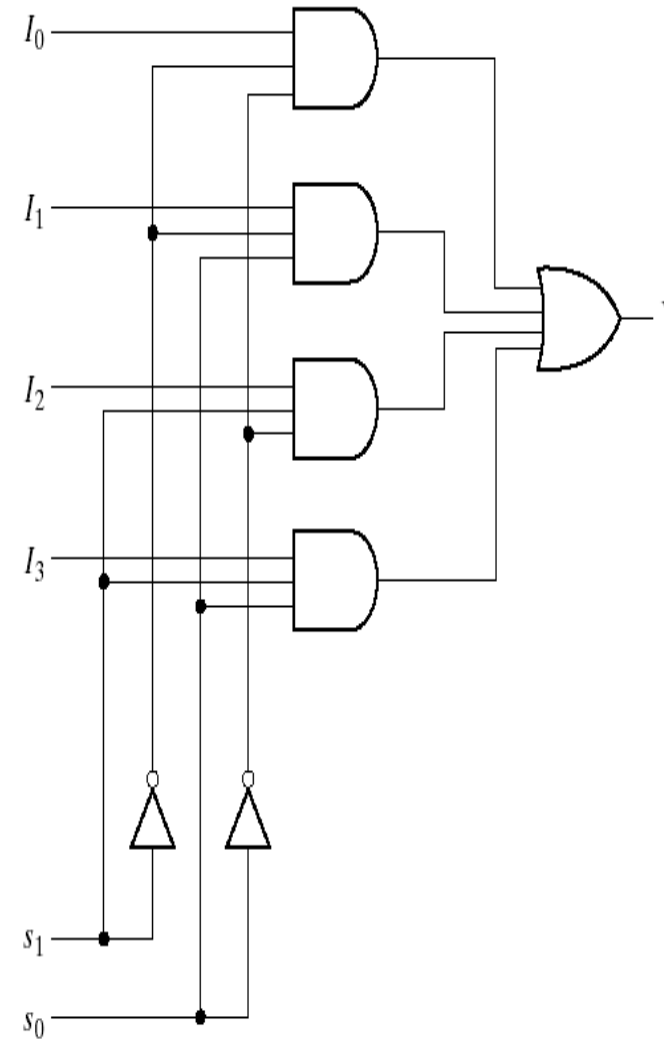
```
            2'b11: y = I3;
```

```
        endcase
```

```
    end
```

```
endmodule
```

Sensitivity list
contains signals
whose change
triggers the
execution of the
block



s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

Procedural Assignments

- Blocking assignments (=)
 - Blocking assignment statements are executed in the order they are specified in a sequential block.
 - A blocking assignment will not block execution of statements that follow in a parallel block.
- Non-blocking statements (<=)
 - Nonblocking assignments allow scheduling of assignments without blocking execution of the statements that follow in a sequential block

Blocking assignment (=)

a = 1 ;

b = a ;

c = b ;



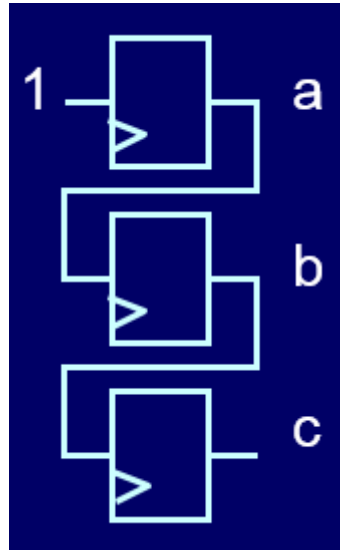
Used to model “**Combinational** ckt.”
a, b, c are nets – not inside always
block

Non-blocking assignment (<=)

a <= 1 ;

b <= a ;

c <= b ;



Used to model “**Sequential** ckt.”
a, b, c are regs – places inside
always block

Race Condition avoidance with nonblocking statements

```
Two concurrent always blocks with blocking statements
always @(posedge clock)
    a = b;
always @(posedge clock)
    b = a;
```

```
Two concurrent always blocks with nonblocking statements
always @(posedge clock)
    a <= b;
always @(posedge clock)
    b <= a;
```

- Always takes the old value of RHS reg
- Order of writes does not matter

Examples

```
1 module block_nonblock();
2 reg a, b, c, d , e, f ;
3
4 // Blocking assignments
5 initial begin
6     a = #10 1'b1; // The simulator assigns 1 to a at time 10
7     b = #20 1'b0; // The simulator assigns 0 to b at time 30
8     c = #40 1'b1; // The simulator assigns 1 to c at time 70
9 end
10
11 // Nonblocking assignments
12 initial begin
13     d <= #10 1'b1; // The simulator assigns 1 to d at time 10
14     e <= #20 1'b0; // The simulator assigns 0 to e at time 20
15     f <= #40 1'b1; // The simulator assigns 1 to f at time 40
16 end
17
18 endmodule
```

Thank You